

An Analysis of Performance Anti-Patterns in Systems Stored on the Brazilian Public Software Portal

E. M. G. D. Santana, R. L. S. D. Alencar and C. A. D. C. Correia

Abstract— The Brazilian Public Software Portal (SPB) provides a social and scientific service through a software repository. Given the importance of this service improvement opportunities are observed through proposals to assess the quality of the software residing on the portal, since it does not have a quality control and auditing for submissions to the software. The aim of this paper is to present existing performance issues in SPB's software by analyzing performance anti-patterns, leading to a process for the collection and analysis of these. The proposed process is the result of an empirical study based on the analysis of data provided by specialized performance measurement tools that aims to contribute to and enrich the knowledge related to performance of stored systems in SPB.

Keywords— Open source, Performance Anti-patterns, Public Brazilian Software Portal, Software Engineering, Software Performance, Performance Evaluation System, Scalability of Web Applications.

I. INTRODUÇÃO

O PORTAL de Software Público Brasileiro (SPB) é um ambiente de compartilhamento de *software open source*. *Open source* é um tipo específico de software que adota um modelo de licença livre e disponibiliza o código-fonte abertamente ao público. A proposta do portal, ao fornecer um espaço de compartilhamento de soluções, resulta em uma gestão de recursos e gastos de informática mais racionalizada, ampliação de parcerias e reforço da política de *software* livre no setor público [1].

Para que o portal possa alcançar os objetivos propostos, é necessário que o software fornecido pelo mesmo seja de qualidade. “Qualidade pode ser definida como o grau em que um sistema, componente ou processo satisfaz os requisitos especificados” [2]. Entretanto, a qualidade de *software* não implica apenas se a funcionalidade do mesmo foi corretamente desenvolvida, mas também depende de atributos não funcionais deste. “Um atributo não-funcional consiste de uma restrição ao serviço ou funcionalidade oferecida pelo sistema” [3]. Dentre estes atributos não-funcionais está o desempenho.

Um software que possui um requisito não-funcional de desempenho de baixa qualidade é impactado por uma variedade de consequências negativas, pois as aplicações não-

performáticas geralmente não entregam seu benefício destinado a uma organização, criando um custo líquido de tempo e dinheiro, e uma perda de elogios de seus usuários [4]. A imagem da organização sofre por conta do baixo desempenho, visto que os usuários associarão a sua baixa qualidade aos produtos da organização mesmo que o problema seja solucionado, e os custos de projeto e o tempo para entrega aumentarão a medida que recursos adicionais sejam alocados para tratar os problemas em desempenho [5].

Uma das causas que acarretam em um *software* de baixa qualidade é o processo de desenvolvimento deste, visto que a qualidade do mesmo é diretamente relacionada com a qualidade de seu processo de desenvolvimento e bons processos são mais suscetíveis de conduzir um *software* de boa qualidade [3]. Entretanto, o Portal de Software Público não possui métodos, ferramentas e procedimentos que realizem um controle de qualidade e auditoria dos *softwares* que são submetidos para o mesmo, e conseqüentemente, não é possível aferir a qualidade das soluções em *software* que residem no portal.

As soluções em *software* fornecidas pelo portal já permitiram informatizar o sistema educacional de escolas municipais, através por exemplo do uso do software público i-Educar, disponibilizado pelo Ministério do Planejamento, Orçamento e Gestão (MP) através Portal de Software Público. Este software tem como objetivo o gerenciamento das ações da rede municipal de ensino e gerou uma economia anual de R\$ 2,4 milhões ao município de Monte Alegre [6]. Esta contribuição social é de fato relevante, entretanto, os benefícios imediatos podem ser suplantados pelas consequências negativas resultantes de problemas referentes a qualidade do software, visto que, como já exposto, o portal não realiza o controle de qualidade e auditoria dos softwares submetidos e por consequência, a qualidade do software implantado nas escolas municipais não é transparente através portal, visto que as carências apresentadas pelo *software* não são conhecidas, e não há uma garantia de que as manutenções evolutivas e corretivas realizadas no *software*, em seu ciclo de vida, manterão o nível de qualidade exigido pelos usuários finais.

Para que o Portal de Software Público mantenha a boa qualidade dos *softwares* que estão e que serão submetidos, se faz necessária a existência de um controle da submissão destes, controle obtido na forma de um processo de auditoria da qualidade do *software*, e inserido neste processo, etapas que auditem a qualidade de desempenho, visto que esta, dado o impacto já apresentado, é de grande importância para a qualidade de um *software*. Entretanto, para que esse processo seja implantado é necessário um levantamento inicial dos problemas dos *softwares* do portal, tais como os problemas em

E. M. G. D. Santana, Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE), Recife, Pernambuco, Brasil, emgs@a.recife.ifpe.edu.br

R. L. S. D. Alencar, Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE), Jaboatão dos Guararapes, Pernambuco, Brasil, roberto.alencar@jaboatao.ifpe.edu.br

C. A. D. C. Correia, Orube Tecnologia, Recife, Pernambuco, Brasil, arthur.carvalho@orubetecnologia.com.br
Corresponding author: R. L. S. D. Alencar

desempenho, para que a etapa de auditoria em desempenho proposta seja construída com base nas carências apresentadas pelos *softwares* já existentes no portal.

Portanto, o objetivo deste trabalho é apresentar problemas de desempenho encontrados por meio da análise dos *softwares* existentes atualmente no SPB. Não são realizados comparativos entre problemas de desempenho ou entre *softwares* do portal, nem são definidos os principais problemas dos *softwares* do SPB, mas são expostas as carências destes no que se refere a qualidade em desempenho e é apresentado o impacto dos problemas mensurados para o usuário final. A partir do levantamento de problemas em desempenho, será possível propor em trabalhos futuros um processo de Controle de Qualidade dos Softwares do portal, através de etapas que auditem a qualidade de desempenho do *software* e que sirvam como critério de aprovação da submissão de um *software* ao Portal de Software Público.

Este trabalho demonstra sua relevância ao contribuir com a qualidade de um *software* do Portal de Software Público Brasileiro, através da identificação e correção de parte dos problemas de desempenho, e consequente incremento da transparência no referido portal a respeito da qualidade do *software* avaliado no estudo de caso.

Especificado na metodologia do trabalho, o processo definido para a identificação de problemas de desempenho não restringe sua importância apenas para guiar o estudo de caso. O entendimento do processo reforça a necessidade da utilização do mesmo ao propor um conjunto de atividades que podem ser utilizadas para compor o processo de controle de qualidade e auditoria dos *softwares* do portal. Assim, o nível de qualidade em desempenho de um *software* que reside no portal pode ser mantido através da correção dos problemas identificados seguindo o processo proposto, e o mesmo pode ser utilizado na avaliação das submissões em outros *softwares* do Portal de Software Público.

Este trabalho possui 5 seções além desta introdução. A seção 2 apresenta uma contextualização para compreensão das subsequentes seções e observações acerca de trabalhos relacionados. Já na seção 3 é apresentada a solução proposta, definindo a metodologia do trabalho. Na seção 4 são apresentados os resultados obtidos, compreendendo um estudo de caso com base na metodologia do trabalho definida. A seção 5 apresenta as conclusões e os trabalhos futuros. Por fim, a seção 6 apresenta os agradecimentos aos colaboradores.

II. REFERENCIAL TEÓRICO

As seções a seguir apresentam uma contextualização para uma maior compreensão dos resultados obtidos e um embasamento teórico sobre os problemas que serão apontados.

A. Mapeamento Objeto-Relacional

O Mapeamento Objeto-Relacional (MOR) provê aos desenvolvedores uma abstração conceitual para mapear registros do banco de dados em objetos. Este mapeamento envolve mapear uma classe em uma tabela do banco de dados, especificando quais atributos correspondem à quais colunas da tabela, e quais os relacionamentos entre as classes e suas

respectivas tabelas. O MOR permite que os desenvolvedores mantenham o foco na lógica do negócio sem se preocupar com os detalhes de acesso a banco de dados. Porém, essa facilidade ao fornecer um conjunto de serviços abstratos para persistência no banco de dados, introduz potenciais problemas de performance [7].

B. Anti-padrões de Performance

Segundo Alexander, “Cada padrão descreve um problema no nosso ambiente e o cerne da solução, de tal forma que você pode usar essa solução mais de um milhão de vezes, sem nunca fazê-lo da mesma maneira” [8]. Dentro desta afirmação, encontra-se os elementos que representam um padrão de projeto: uma solução para um problema comum em um determinado contexto [9].

Padrões de projeto e anti-padrões estão relacionados. Um anti-padrão é originado da aplicação de um padrão em um contexto equivocado, resultado da falta de conhecimento ou experiência na tentativa de resolver um tipo de problema. Porém, diferente de um padrão de projeto, que é representado por uma solução diante da resolução de um problema, um anti-padrão possui duas soluções. A primeira é problemática e representa a essência do anti-padrão: uma solução para um problema que gera consequências negativas. A segunda é conhecida como: *Refactored Solution*, e descreve como resolver a solução problemática [10]. Anti-padrões e padrões de projeto não destacam explicitamente o impacto no desempenho do software. Entretanto, os anti-padrões definidos por Smith e Williams [11-14] destacam o impacto desses problemas no desempenho de um *software*.

C. Métricas

Para que sejam apontados problemas de desempenho nos *softwares* do SPB, se faz necessário a definição de uma métrica para mensurar o impacto de um problema de desempenho. Considerando que: “*Performance é o grau em que um componente ou sistema realiza suas funções designadas dentro de determinadas restrições, tais como velocidade, precisão ou uso de memória, impostas*” [2]. Como mensurar esse grau? Para isso, são utilizados indicadores-chave de desempenho (KPI). Esses indicadores são divididos em dois tipos: orientados a serviço e orientados a eficiência.

Os indicadores orientados a serviço medem o quão bem (ou não) uma aplicação fornece um serviço para os usuários finais. Um dos indicadores orientados a serviço é o tempo de resposta. “*O tempo de transação ou tempo de resposta é o tempo que a aplicação leva para responder a requisição de um usuário*” [4]. No que se refere a orientação a serviço, o tempo de transação é uma das mais importantes características do desempenho de um sistema da perspectiva de um usuário [15]. Esse indicador será utilizado para medir o impacto de um problema elicitado de um sistema do portal de software público, visto que oferecer um serviço de alta qualidade aos usuários é um objetivo do SPB.

II. SOLUÇÃO PROPOSTA

A. Contexto do Levantamento de Problemas

Considerando o amplo conteúdo que compreende a área de estudo do desempenho de sistemas e a variedade dos sistemas que se encontram no portal, se faz necessário a definição de um tema, que afete significativamente a métrica do tempo de resposta, para guiar o levantamento de problemas de desempenho dos *softwares* do portal. Visto que as atividades de entrada/saída são custosas para o desempenho de uma aplicação no que se refere ao tempo de resposta, e que essas atividades são realizadas no processo de persistência dos dados da mesma, normalmente em um banco de dados, já que estes são um componente essencial da vida na sociedade moderna [16], é adequado definir como tema as atividades de entrada/saídas realizadas na persistência em banco de dados das aplicações.

Definida a métrica e o contexto em que serão levantados os problemas de desempenho dos *softwares* do portal, será especificado qual o tipo de problema dentro desse contexto que podemos mensurar pela métrica estabelecida. Considerando a relevância do problema para etapas de auditoria de software, é importante que o problema de desempenho seja estruturado, possua sua definição bem estabelecida na literatura e consequentemente seja fácil de se rastrear, de forma que possíveis soluções sejam sugeridas. Essas características desejadas apresentam-se em um anti-padrão. “O conceito de anti-padrão é semelhante ao de um padrão, exceto que em vez de uma solução ele lhe dá algo que parece superficialmente como uma solução, mas que não é uma” [17], e um anti-padrão de performance descreve esta “solução equivocada” que tem um impacto significativo e degradante na performance. Portanto, serão elicitados anti-padrões de performance inseridos no contexto de persistência dos dados em banco de dados e avaliados pela métrica do tempo de resposta.

B. Cenário de estudo

A metodologia proposta será através da definição de um estudo de caso. Este procedimento supõe que se pode adquirir conhecimento do fenômeno estudado a partir da exploração extensa de um único caso [18]. O estudo de caso será realizado no Sistema de Gestão Comercial de Gás Natural (GGAS), este encontra-se disponível e possui acesso fácil ao seu repositório por meio do Portal de Software Público Brasileiro e foi selecionado para o estudo de caso por ser um sistema de grande porte no que se refere a tamanho do sistema e quantidade de usuários que os utilizam.

De acordo com informações fornecidas pela Mitsui Gás e Energia do Brasil, que possui participação acionária nas distribuidoras, o GGAS ainda não está em produção, mas quando for concluído a implantação poderá ser acessado por aproximadamente 150.000 usuários (número aproximado de clientes de todas as distribuidoras que estão realizando a implantação) com um custo de aproximadamente R\$ 2.000.000 para ser desenvolvido, possuindo mais de 295.598

linhas de código, e R\$ 6.500.000 para ser implantado nas companhias distribuidoras de gás natural dos estados de Alagoas, Sergipe, Bahia, Paraná, Paraíba e Rio Grande do Norte.

C. Processo do Levantamento de Problemas

Para este será realizado um *profiling*, seguido de uma inspeção nos módulos do sistema definido para o estudo de caso. Na engenharia de software, *profiling* é um forma de análise dinâmica da aplicação, ou seja, a análise desta em tempo de execução, que mede, por exemplo, a memória, complexidade de tempo do programa, o uso de instruções em particular (instruções em SQL, por exemplo), duração e frequência de chamadas de função [19]. Coletadas as informações do *profiling*, o código-fonte do módulo estudado será inspecionado manualmente.

A etapa de inspeção tem como objetivo utilizar as informações obtidas com o *profiling* para auxiliar na identificação dos anti-padrões de performance ao verificar qual parte do código-fonte apresenta um potencial anti-padrão. Essa identificação do anti-padrão é obtida através das características descritas no problema definido pelo anti-padrão que correspondem com o conjunto de informações obtidos na inspeção e no *profiling*.

Identificado um anti-padrão de performance, será mensurado o tempo de resposta do módulo ou funcionalidade a qual o anti-padrão está inserido, para se obter quanto do tempo de resposta de uma funcionalidade ou módulo da aplicação é impactado pela ocorrência deste problema de performance. Para o caso de uma proposta de otimização, será apresentada também a redução de impacto obtida.

D. Ferramentas e Ambiente

No que compreende as ferramentas, para inspeção manual é válido qualquer editor de código-fonte ou IDE e para o *profiling* será utilizada a ferramenta XRebel. O XRebel, é uma ferramenta para análise de dinâmica de aplicações Java Web, que permite analisar a atividade de banco de dados da aplicação instrumentada, medindo o tempo de resposta de cada requisição a um sistema web [20]. As características que esta ferramenta apresenta é ideal para o levantamento dos anti-padrões na aplicação proposta para o estudo de caso, visto que esta é uma aplicação Java Web.

III. RESULTADO DOS ESTUDOS

E. Estudo de Caso

Os anti-padrões catalogados por Smith e Williams [11-14], abordam uma variedade de indicadores de qualidade não-funcional. Os anti-padrões que serão abordados no estudo de caso localizam-se dentro do contexto definido para o levantamento de problemas e são relevantes para a métrica proposta. Porém, apesar do processo de levantamento dos problemas abordar esta metodologia, não é desconsiderada a aplicação prática e a relevância dos anti-padrões não apresentados.

Dada a grandeza do sistema escolhido para o estudo de caso, como exposto no cenário de estudo, é necessário definir quais módulos serão avaliados. Para isso, foi realizado um estudo no mesmo, exercitando as funcionalidades do sistema ao realizar atividades de um usuário final. Ficou definido que serão instrumentados os módulos de *login* e contrato. O *login* foi escolhido por ser uma funcionalidade obrigatória para o uso do sistema e o de contrato por ser considerado um dos maiores módulos da aplicação, uma vez que este se relaciona com praticamente todos os outros módulos.

1) Instrumentação no Módulo de Acesso

Durante o processo de levantamento de problemas realizado no módulo de acesso, foi identificado a ocorrência do anti-padrão *Unnecessary Processing* na validação do login do usuário. Este anti-padrão se caracteriza pela existência de processamento que é executado, mas que não é necessário [13].

Para validar o *login* é necessário consultar um registro do usuário no banco de dados. Este registro é mapeado em um objeto e possui associações com outros objetos também mapeados em registros de outras tabelas. Entretanto, nenhum desses objetos associados, que são consultados com o objeto usuário, são utilizados para validação do *login*, e o objeto que representa o usuário nem sequer sai do escopo da funcionalidade de validar usuários, ou seja, ele é carregado do banco de dados e não é utilizado após a validação do *login*.

Utilizando o tempo de resposta fornecido pela ferramenta XRebel, foi possível realizar um comparativo, demonstrado na Tabela I, que apresente o impacto desta consulta na funcionalidade que valida o *login* de um usuário.

TABELA I
IMPACTO DA CONSULTA POR UM USUÁRIO NA FUNCIONALIDADE DE VALIDAÇÃO

Tempo de Resposta da Validação de um Usuário	Duração da consulta por um usuário	Impacto da consulta no tempo de resposta da funcionalidade
14.9 ms	13.5 ms	90.60%

Para solucionar o problema, a solução que é recomendada na definição anti-padrão *Unnecessary Processing* é que seja eliminado o processamento desnecessário. Como o GGAS utiliza mapeamento objeto-relacional poderíamos alterar o mapeamento da classe que representa o usuário, visto que esse mapeamento define quais classes relacionadas a classe que representa o usuário também serão consultadas no banco quando for requisitado o registro de um usuário. Entretanto, alterar o mapeamento pode resultar em “quebra” de funcionalidades ou impactos negativos no desempenho em outros módulos da aplicação. Portanto, foi implementado um novo método para consultar um usuário utilizando o seu *login*, mas consultando apenas os atributos necessários para validar um *login* de usuário.

Alterando a funcionalidade, fazendo-a utilizar o método otimizado para consultar os usuários, obtemos uma redução significativa no impacto da consulta ao banco, demonstrada na

Tabela II.

TABELA II
IMPACTO DA CONSULTA OTIMIZADA NA FUNCIONALIDADE DE VALIDAÇÃO

Tempo de Resposta da funcionalidade não otimizada	Tempo de resposta da funcionalidade após otimização	Redução obtida no tempo de resposta da funcionalidade
14.9 ms	1.4 ms	90.60%

No GGAS, após a autenticação do usuário é realizada a autorização. Esta funcionalidade é baseada nos conceitos de: quais módulos do sistema o usuário tem permissão de acesso, onde estes agrupam recursos do sistema, e as operações sobre esses recursos que o usuário pode efetuar. Durante o processo de *login* do usuário, é consultado no banco de dados os módulos, os recursos e as operações. Para cada recurso é realizada uma consulta no banco de dados, onde cada consulta possui estrutura lógica idêntica. A estratégia de consulta descrita, apresenta as características de um anti-padrão, conhecido como *Empty Semi Trucks*. Esse anti-padrão ocorre quando um número excessivo de requisições são necessárias para executar uma tarefa, que poderia ser executada em apenas uma requisição [12]. Esta requisição pode ser interpretada como uma recuperação de registros de um banco de dados.

De acordo com o processo estabelecido na metodologia, deve-se após a identificação do problema, através da ferramenta, inspecionar o código. Durante a inspeção foi observado que duas funcionalidades consultam os recursos do sistema. É apresentado na Tabela III um comparativo que demonstra o impacto das funcionalidades no tempo de resposta do *login*.

TABELA III
IMPACTO DAS FUNCIONALIDADES NO MÓDULO DE ACESSO

Tempo de Resposta médio do login	Tempo de resposta total das funcionalidades	Impacto no tempo de resposta do login
3.9 s	1.2 s	30.76%

Para a resolução do problema apontado, o anti-padrão *Empty Semi Trucks* determina que deve-se agrupar as requisições em lote, o *framework* de mapeamento objeto-relacional utilizado pelo GGAS provê uma forma simples de trabalhar com consultas de registros em lote, alterando o mapeamento da classe, mas sem risco de impacto negativo na performance ou “quebra” de outras funcionalidades. As consultas nas funcionalidades que envolviam a consulta de recursos foram agrupadas em lote, obtendo-se uma redução no número destas e consequentemente no tempo de resposta das funcionalidades, resultando no cenário apresentado na Tabela 4.

TABELA IV
OTIMIZAÇÕES NAS FUNCIONALIDADES DE AUTORIZAÇÃO DO
MÓDULO DE ACESSO

	Tempo de Resposta sem otimização	Tempo de resposta com otimização	Redução do tempo de resposta
Listagem de Módulos	872.8 ms	496.1 ms	43.15%
Listagem de Recursos	324.9 ms	44.9 ms	86.18%

2) Instrumentação do Módulo de Contrato

Um contrato no GGAS possui um modelo, este define a estrutura do contrato, as seções que compõem o mesmo (Abas) e os atributos destas. A Fig. 1 apresenta uma parte do digrama de classes, com as mais relevantes, resultante da inspeção realizada no código do módulo de contrato.

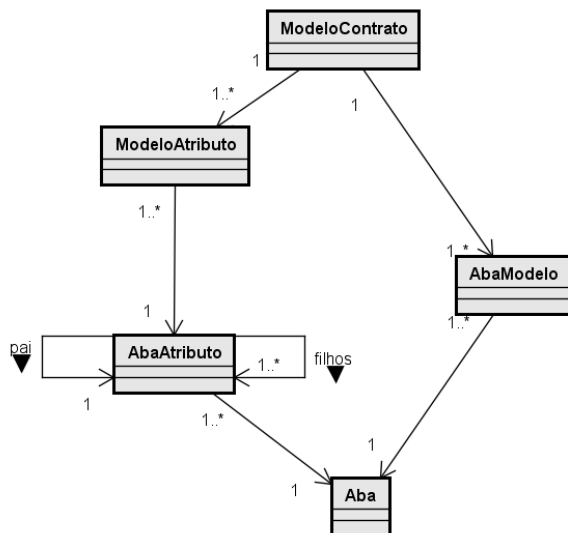


Figura 1. Diagrama de Classes do Modelo de Contrato. Fonte: Elaborada pelo autor.

Quando é necessário obter as Abas de um Modelo de Contrato, deve-se obter uma coleção de objetos da classe denominada AbaModelo e partir destes obtemos os objetos da classe Aba. Este procedimento se caracteriza por uma das instâncias do anti-padrão *Circuitous Treasure Hunt*, este ocorre quando é necessário consultar em vários lugares para se obter um “resultado final”[11].

A classe AbaModelo mapeia uma tabela de junção entre a classe ModeloContrato e a classe Aba. Este mapeamento é desnecessário, visto que poderia ser estabelecido um relacionamento muitos para muitos entre ModeloContrato e Aba sem a necessidade da classe AbaModelo. Neste cenário atual, é adicionado um custo de processamento das consultas e da criação de instância dos objetos para os resultados intermediários, como o resultado dos objetos da classe AbaModelo. Para a solução, o anti-padrão propõe a criação de

uma associação direta com o “resultado final”.

A Tabela 5 apresenta um cenário comparativo considerando que, com a relação direta entre um modelo de contrato e suas abas, as mesmas são obtidas em uma única consulta ao banco.

TABELA V
CENÁRIO COMPARATIVO DA CONSULTA POR ABAS DE UM
MODELO

Tempo de Resposta médio sem otimizações	Tempo de resposta médio após otimizações	Redução do tempo de resposta
55.2 ms	2.5 ms	95.47%

Para incluir um novo contrato, é necessário consultar um cliente para associá-lo ao contrato. Inspeccionando a estratégia de consulta de clientes foi observado que a mesma é não escalável e cresce linearmente a quantidade de clientes cadastrados na aplicação, e para cada subconjunto de clientes que o usuário solicita para apresentação, é consultado o conjunto completo dos clientes, de acordo com o filtro de pesquisa especificado pelo usuário utilizando a aplicação. A estratégia de consulta apresentada é característica do anti-padrão *The Ramp*, este afirma que: “a performance é impactada em qualquer situação em que a quantidade de processamento necessário para atender a uma solicitação aumenta com o tempo”[13]. Para resolver o problema, alteramos o algoritmo de consulta, realizando uma paginação em nível de banco de dados. Obteve-se uma redução significativa no tempo de resposta da consulta, resultando no cenário apresentado na Tabela 6, com 9834 clientes cadastrados na aplicação.

TABELA VI
CENÁRIO COMPARATIVO DA CONSULTA POR CLIENTES NO
MÓDULO DE CONTRATO

Tempo de Resposta médio sem otimizações	Tempo de resposta médio após otimização	Redução do tempo de resposta
31.4 s	1.2 s	96.17%

IV. CONCLUSÃO

O processo definido para levantamento de problemas obteve resultados satisfatórios para o cenário analisado no estudo de caso. Utilizar anti-padrões como um referencial para a pesquisa de problemas se mostrou uma abordagem assertiva, pois as características bem definidas de um anti-padrão, suportadas pela ferramenta de *profiling*, permitiram identificar um anti-padrão de forma rápida e objetiva, visto que as informações fornecidas pela ferramenta permitiram que após a localização do anti-padrão, se identificasse a parte do código onde a inspeção se fazia necessária.

Dentre os problemas levantados no estudo de caso, os anti-padrões: *Unnecessary Processing* e *Empty Semi Trucks* apareceram com mais frequência. Durante a inspeção, foi observado que uma das possíveis causas para ocorrência dos

problemas classificados como *Unecessary Processing* é a reutilização não controlada de código, visto que os problemas são resultantes da reutilização de funcionalidades que possuem códigos não coesos e na maioria das vezes desnecessários.

Como trabalhos futuros, pode-se utilizar o processo definido para o levantamento de problemas de desempenho em diferentes *softwares* armazenados no SPB, estendendo e ampliando o catálogo de anti-padrões abordados, visto que os mesmos podem ser utilizados em diferentes contextos de problemas e são relevantes para diversificados indicadores de qualidade não-funcional. Além disso, pode ser desenvolvido um software especialista que ajude o desenvolvedor a mapear, controlar e corrigir os anti-padrões, mapeando assim o processo através da ferramenta.

Considerando as relevâncias científica do assunto é importante apontar que o estudo de anti-padrões abordado neste trabalho, ainda não é explorado de forma abrangente na literatura no contexto de melhoria de performance. E apresentar o uso real dos anti-padrões pode auxiliar na percepção que, de fato, estes são extremamente frequentes nas aplicações atuais e são demasiados degradantes para a qualidade de uma aplicação.

Os problemas abordados no estudo de caso ajudam a preencher a lacuna de conhecimento sobre o estado atual em que se encontram alguns dos sistemas hospedados atualmente no Portal de Software Público Brasileiro, no quesito qualidade em desempenho, e por consequência, favorecem o planejamento de um processo de controle de qualidade e auditoria de software, processo este que poderia contribuir para aumentar a qualidade dos softwares armazenados no Portal.

AGRADECIMENTOS

O presente trabalho foi realizado com apoio do Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE).

REFERÊNCIAS

- [1] P. Meireles. Sobre o portal. Disponível em: <<https://softwarepublico.gov.br/social/spb/sobre-o-portal>>. Acesso em: 23 ago. 2015.
- [2] IEEE Standard Glossary of Software Engineering Terminology, IEEE Standard 610.12-1990.
- [3] I. Sommerville, *Engenharia de software*. São Paulo: Pearson Prentice Hall, 2011.
- [4] I. Molyneaux, *The art of application performance testing*. Sebastopol, CA: O'Reilly Media, 2009.
- [5] C. Smith and L. Williams, *Performance solutions: A practical guide to creating responsive scalable software*. Boston, Mass.: Addison-Wesley, 2002.
- [6] P. Brasil. RN: Uso de software público gera economia de R\$ 2, 4 mi. Disponível em: <<http://www.brasil.gov.br/economia-e-emprego/2015/09/rn-uso-de-software-publico-gera-economia-de-r-2-4-mi>>. Acesso em: 24 ago. 2015.
- [7] T.-H. Chen, W. Shang, Z. M. Jiang, A. E. Hassan, M. N. Nasser, and P. Flora, "Detecting performance anti-patterns for applications developed using object-relational mapping," in ICSE, pp. 1001–1012, 2014.
- [8] C. Alexander, S. Ishikawa and M. Silverstein, *A pattern language*. New York: Oxford University Press, 1977.
- [9] E. Gamma, *Padrões de projeto*. Porto Alegre: Bookman, 2000.
- [10] W. J. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, *AntiPatterns: refactoring software, architectures, and projects in crisis*. New York: John Wiley & Sons Inc., 1998.

- [11] C. U. Smith and L. G. Williams, "Software performance antipatterns," in Workshop on Software and Performance, pp. 127–136, 2000.
- [12] C. U. Smith and L. G. Williams, "More new software antipatterns: Even more ways to shoot yourself in the foot," in Int. CMG Conference. Computer Measurement Group, pp. 717–725, 2003.
- [13] C. U. Smith and L. G. Williams, "New software performance antipatterns: More ways to shoot yourself in the foot," in Int. CMG Conference. Computer Measurement Group, pp. 667–674, 2002.
- [14] C. U. Smith and L. G. Williams, "Software performance antipatterns; common performance problems and their solutions," in Int. CMG Conference. Computer Measurement Group, pp. 797–806, 2001.
- [15] L. Grinshpan, *Solving enterprise applications performance puzzles*. Hoboken, N.J.: Wiley-IEEE Press, 2012.
- [16] R. Elmasri, S. B. Navathe, and D. Vieira, *Sistemas de banco de dados*, 6th ed. São Paulo (SP): Pearson Addison Wesley, 2011.
- [17] L. Rising, *The patterns handbook*. Cambridge, U.K.: Cambridge University Press, 1998.
- [18] M. M. Ventura, "O estudo de caso como modalidade de pesquisa," *Revista da Sociedade de Cardiologia do Estado do Rio de Janeiro*, Rio de Janeiro, vol. 20, pp. 383–386, 2007.
- [19] E. Coppa, C. Demetrescu, and I. Finocchi, "Input-sensitive profiling," *IEEE Trans. Software Eng.*, vol. 40, no. 12, pp. 1185–1205, 2014.
- [20] S. Prakash. ZeroTurnaround Releases New Version of its Lightweight Java Profiler, XRebel. Disponível em: <<https://zeroturnaround.com/rebellabs/zeroturnaround-releases-new-version-of-its-lightweight-java-profiler-xrebel>>. Acesso em: 10 Set. 2015.



Edmilson Manoel Guilherme de Santana, é graduando em Análise e Desenvolvimento de Sistemas e pesquisador pelo Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE). Seu foco de trabalho e pesquisa envolve: Arquitetura de Software, Avaliação de Desempenho e Sistemas e Escalabilidade de Aplicações WEB. <http://lattes.cnpq.br/3168545314234702/>



Roberto Luiz Sena de Alencar, é graduado no curso superior de Análise e Desenvolvimento de Sistemas pela Faculdade de Tecnologia IBRATEC (2007) e mestre em Engenharia de Software pelo C.E.S.A.R – Centro de Estudos e Sistemas Avançados do Recife (2010). É professor e pesquisador do Instituto Federal de Educação, Ciência e Tecnologia de Pernambuco (IFPE) desde 2014. Possui mais de 10 anos de experiência adquiridos em empresas de médio e grande porte do Porto Digital atuando em atividades de desenvolvimento, coordenação e gerência de TI. Seu foco de trabalho e pesquisa envolve: Metodologias Ágeis de Software, Reuso de Software, Testes de Software, Avaliação de Desempenho e Sistemas e Escalabilidade de Aplicações WEB. <http://lattes.cnpq.br/4839735568204936>



Carlos Arthur de Carvalho Correia, é bacharel em Ciências da Computação pela Universidade Federal de Alagoas (UFAL), especialista em Engenharia de Software C.E.S.A.R – Centro de Estudos e Sistemas Avançados do Recife e MSc em Engenharia de Software pelo C.E.S.A.R. Possui experiência em docência como professor de Ciências da Computação pela FG – Faculdade Guararapes por 6 anos. Seu foco de trabalho é Arquitetura de Software, sistemas distribuídos e escalabilidade.